

AUTO ENCODER

INTERNSHIP REPORT

Submitted in partial fulfillment of requirements to
RVR & JC COLLEGE OF ENGINEERING

By

N. Lakshman with Y20CS119



APRIL-JUNE 2022

R.V.R & J.C COLLEGE OF ENGINEERING(Autonomous)

(Affiliated to Acharya Nagarjuna University)

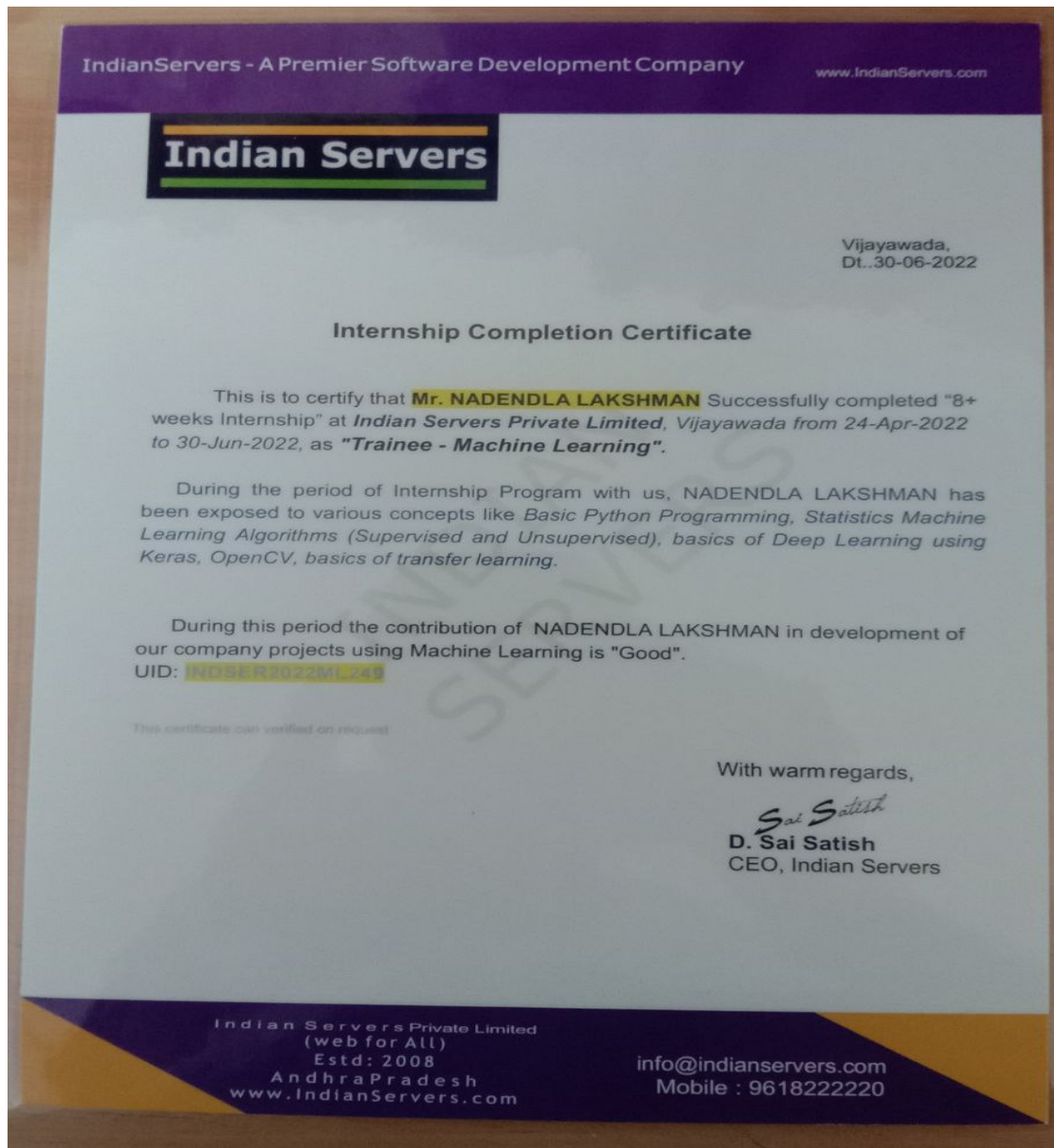
Chandramoulipuram :: Chowdavaram

GUNTUR - 522019

R.V.R & J.C COLLEGE OF ENGINEERING

(Autonomous)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING CERTIFICATE



ACKNOWLEDGEMENT

The successful completion of any task would be incomplete without a proper suggestions , guidance and environment. combination of these three factors acts like backbone to our “**AUTO ENCODER**”

We express our sincere thanks to **Dr. M. Sreelatha**, Head of the Department of Computer Science and Engineering for her encouragement and support to carry out this Project successfully.

We are very much thankful to **Dr. K. Ravindra**, Principal of RVR &JC College of Engineering, Guntur for providing this supportive Environment.

Finally, we submit our reserves thanks to lab staff in Department of Compute Science and Engineering and to all our friends for their cooperation during the preparation.

N.Lakshman (Y20CS119)

ABSTRACT

The concept of auto encoder was originally proposed by Le Cun in 1987, early works on auto encoder were used for dimensionality reduction or feature learning. Recently, with the popularity of deep learning research, auto encoder has been brought to the forefront of generative modeling.

Requirements engineering is the first phase in software development life cycle and it also plays one of the most important and critical roles. Requirement document mainly contains both functional requirements and non-functional requirements. Non-functional requirements are significant to describe the properties and constraints of the system. Early identification of Non-functional requirement has direct impact on the system architecture and initial design decision. Practically, non-functional requirements are extracted manually from the document. This makes it tedious, time-consuming task and prone to various errors. In this paper, we propose an automatic approach to identify and classify non-functional requirements using semantic and syntactic analysis with machine learning approaches from unconstrained documents. We used A dataset of public requirements documents (PURE) that consists of 79 unconstrained requirements documents in different forms. In our approach, features were extracted from the requirement sentences using four different natural language processing methods including

statistical and state-of-the-art semantic analysis presented by Google word2vec and bidirectional encoder representations from transformers models. The adopted approach can efficiently classify non-functional requirements with an accuracy between 84% and 87% using statistical vectorization method and 88% to 92% using word embedding semantic methods. Furthermore, by fusing different models trained on different features, the accuracy improves by 2.4% compared with the best individual classifier.

Here we present an inclusive review of recent and successful content-based techniques. Our focus is primarily on machine learning-based image recognition and variants that are inspired by them. I report on related ideas, techniques, major efforts and cutting-edge art in the field. Preliminary interpretation of prior work shows the basics of deep learning. In this I conclude by studying techniques, methods, evaluation criteria, and explore promising apprehensions of the latest developments and suggest future lines of investigation.

There are 4 hyper parameters that we need to set before training an auto encoder: Code size: It represents the number of nodes in the middle layer. Smaller size results in more compression. Number of layers: The auto encoder can consist of as many layers as we want.

Key concepts of Auto Encoder are: Encoding, Bottle Neck and Decoding, Deep CNN, Image De noising and Simple Auto encoder.

Auto encoder is an unsupervised artificial neural network that learns how to efficiently compress and encode data then learns how to reconstruct the data back from the reduced encoded representation to a representation that is as close to the original input as possible.

Auto encoders train the network to explain the natural structure in the data into efficient lower-dimensional representation. It does this by using decoding and encoding strategy to minimize the reconstruction error. The input and the output dimension have 3000 dimensions, and the desired reduced dimension is 200.

Contents

Title	i
Certificate	ii
Acknowledgement	iii
Abstract	iv
Contents	iv
List of Figures	vii
List of Abbreviations	viii

1 Introduction	Page No
----------------	---------

- 1.1Background.....
- 1.2Problem statement.....
- 1.3Significance of the work.....

2 SYSTEMANALYSIS

- 2.1 Requirements Specification
 - 2.1.1 Functional Requirements
 - 2.1.2 Non-Functional Requirements

3 SYSTEMDESIGN

- 3.1 Architecture of the proposed system
- 3.2 Flow of work

.....

.....

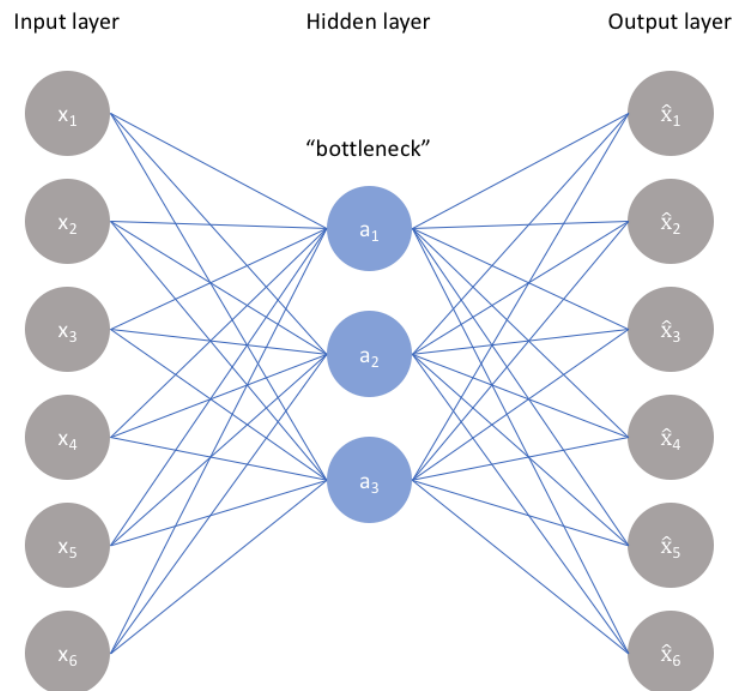
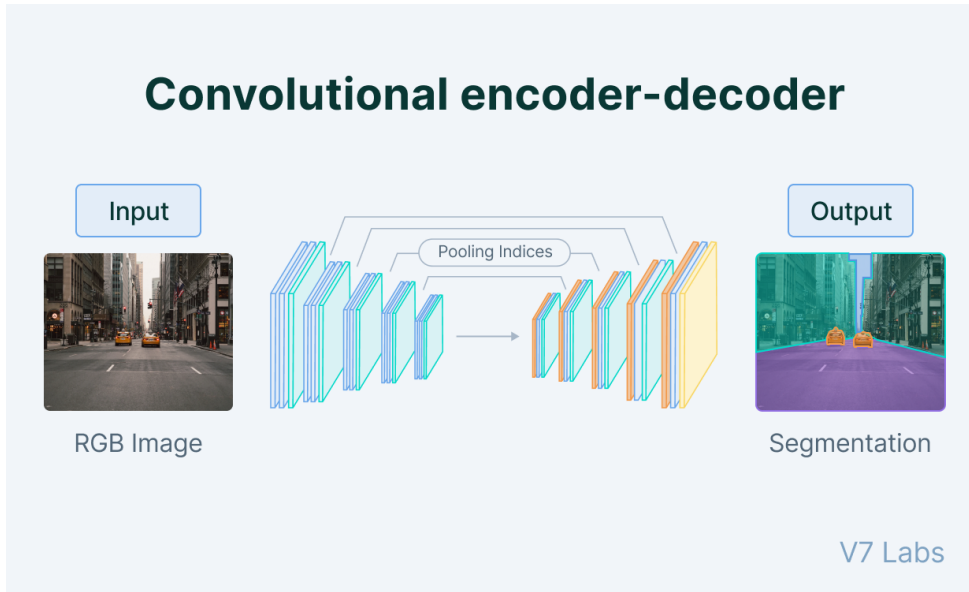
4 Testing and Implementation

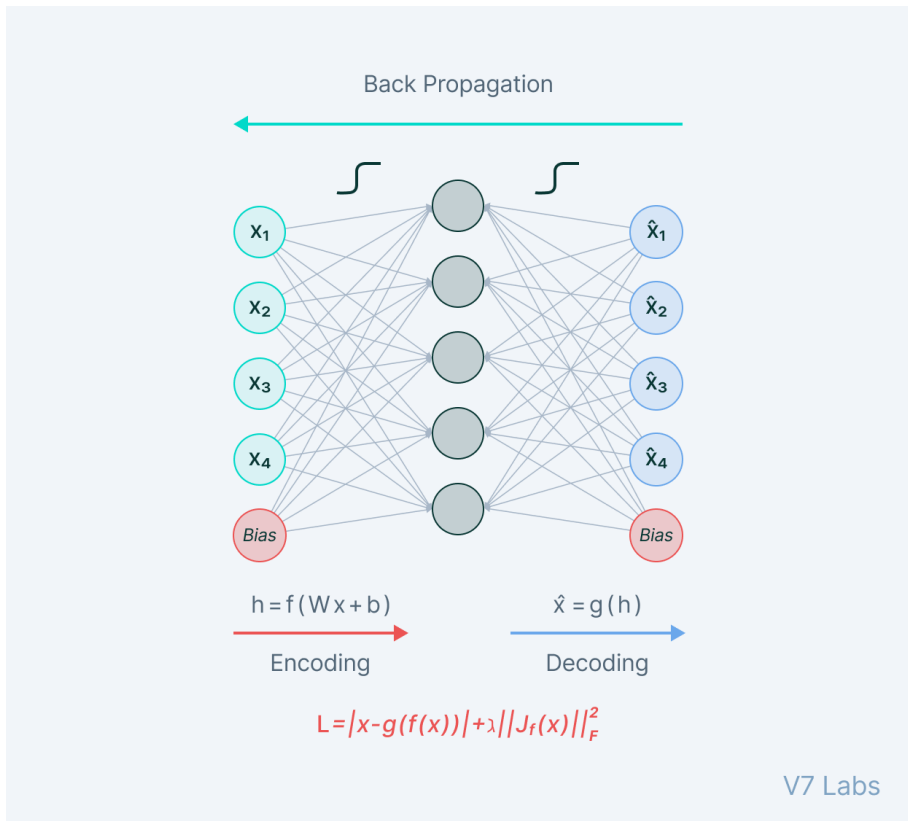
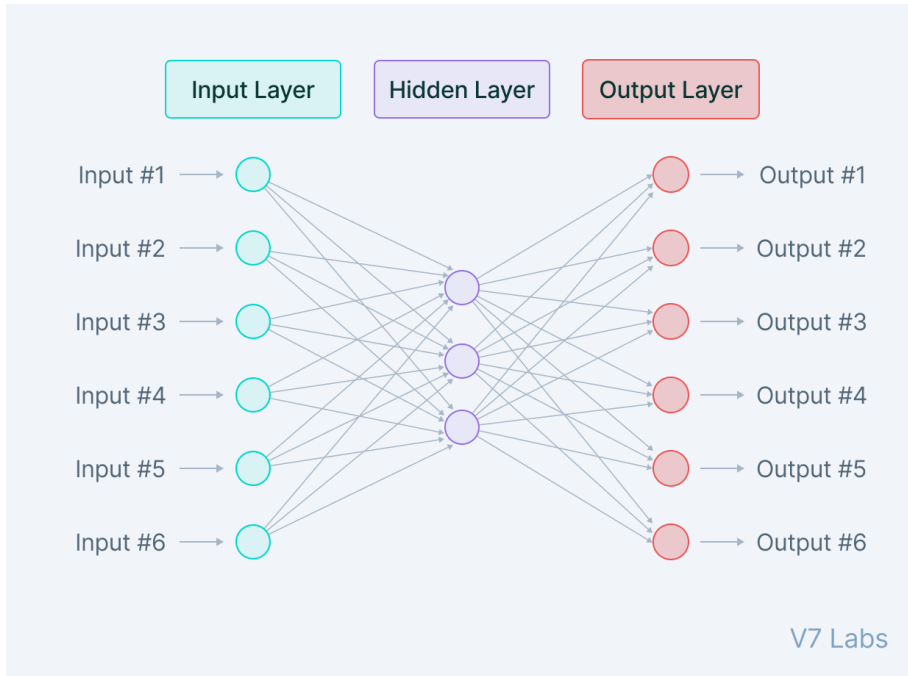
5 RESULTS

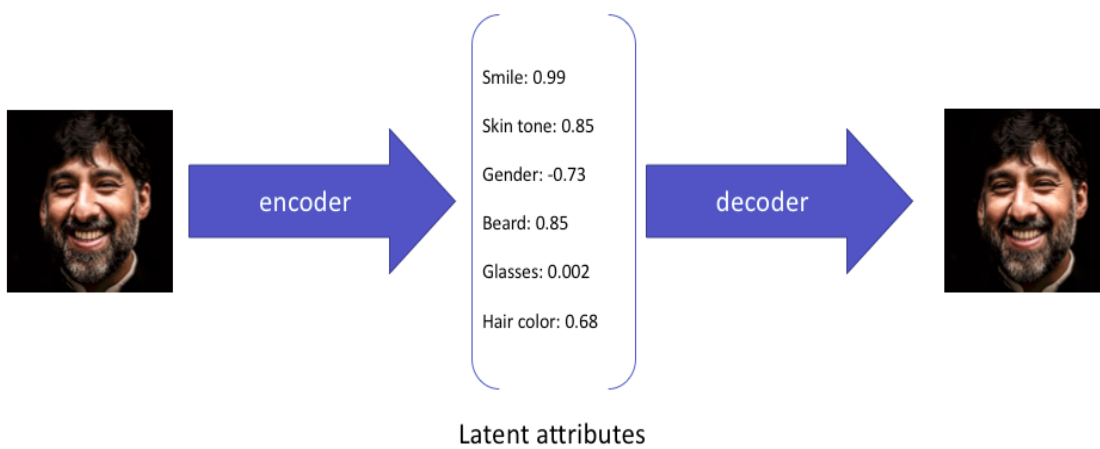
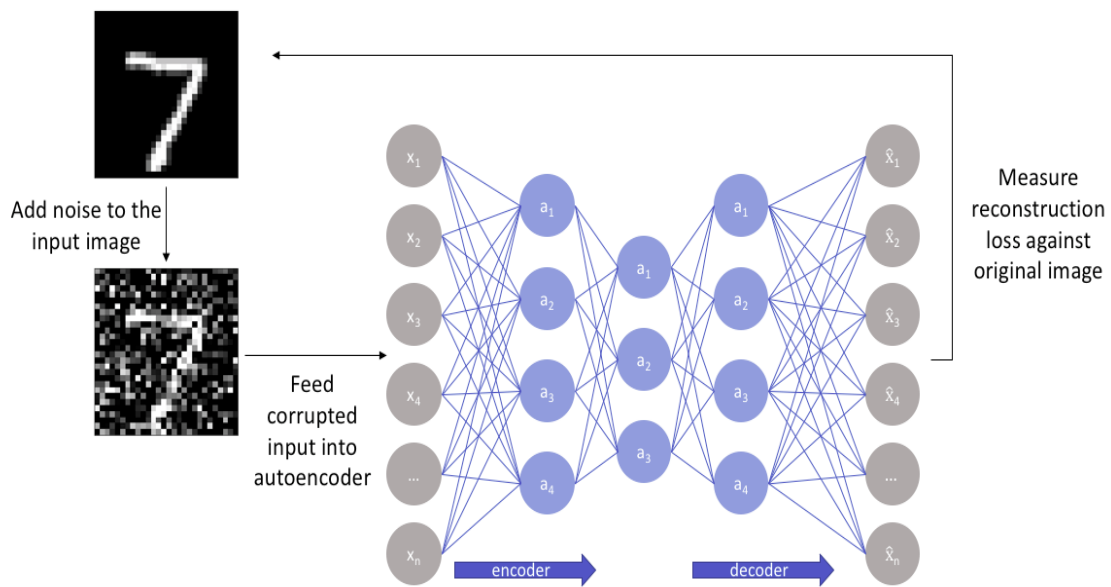
6 CONCLUSION AND FUTURE WORK

7 REFERENCES

List of Figures :

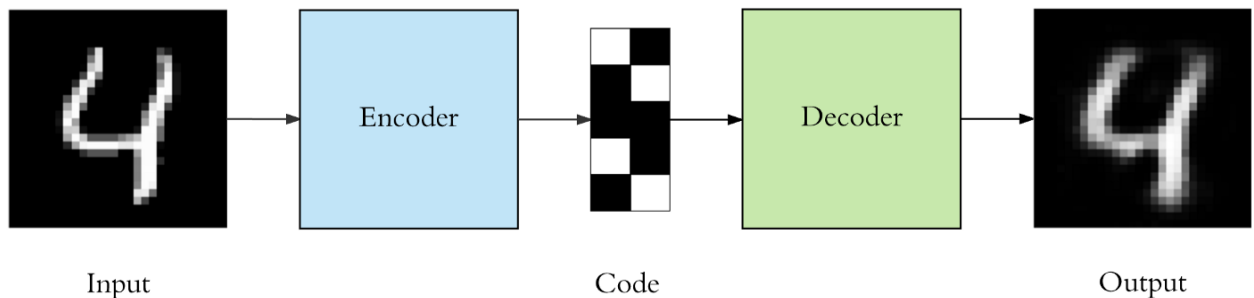






Introduction:

Auto encoders are neural network-based models that are used for unsupervised learning purposes to discover underlying correlations among data and represent data in a smaller dimension. The auto encoders frame unsupervised learning problems as supervised learning problems to train a neural network model.



To build an auto encoder we need 3 things: an encoding method, decoding method, and a loss function to compare the output with the target. We will explore these in the next section.

1.1. Background:

An auto encoder is an unsupervised learning technique for neural networks that learns efficient data representations (encoding) by training the network to ignore signal “noise.” Auto encoders can be used for image de noising, image compression, and, in some cases, even generation of image data.

The auto encoder consists of two parts: the encoder and the decoder. The encoder learns how to interpret the input and compress it to an internal representation defined by the bottleneck layer. The decoder takes the output of the encoder (the bottleneck layer) and attempts to recreate the input.

Application of this method, doesn't shows any promising results because the rules should be necessary. Constantly updating the rules and methods just causes waste of time and requires more maintenance. As compared to knowledge Engineering, Machine learning is more appropriate approach.

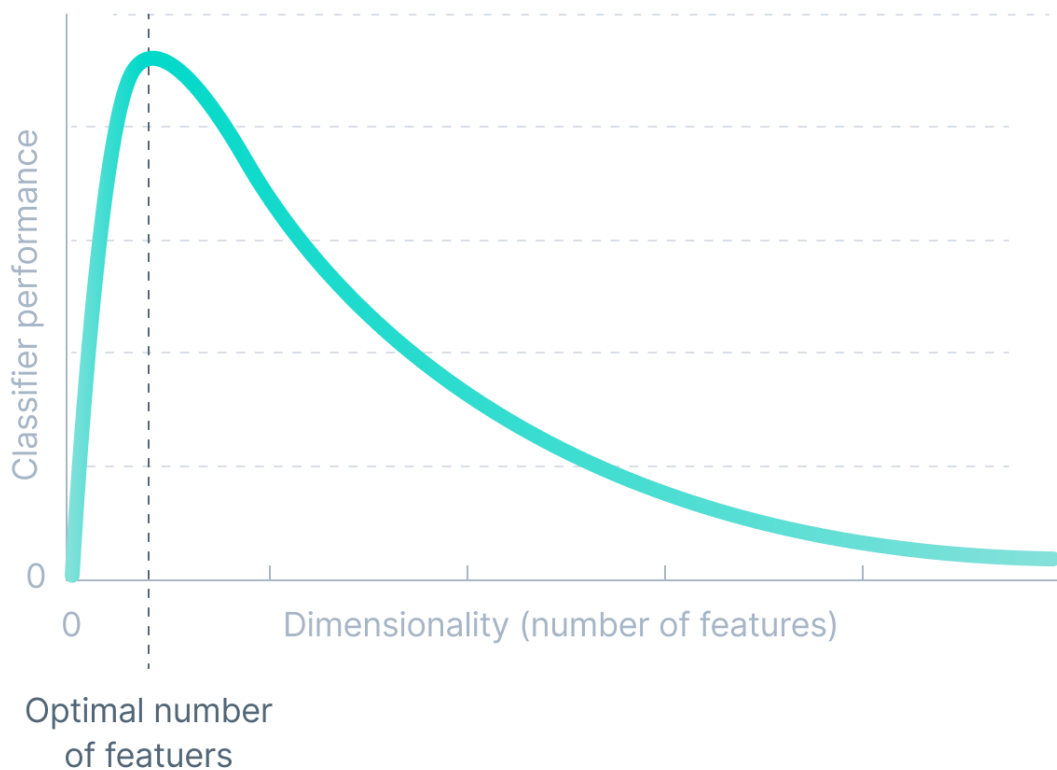
1.2. Problem Statement:

The aim of an auto encoder is to learn a lower-dimensional representation (encoding) for a higher-dimensional data, typically for dimensionality reduction, by training the network to capture the most important parts of the input image.

The goal is to get an output identical with the input. Note that the decoder architecture is the mirror image of the encoder. This is not a requirement but it's typically the case. The only requirement is the dimensionality of the input and output needs to be the same. Anything in the middle can be played with.

1.3. Significance of the Work:

Put simply, auto encoders are used to help reduce the noise in data. Through the process of compressing input data, encoding it, and then reconstructing it as an output, auto encoders allow you to reduce dimensionality and focus only on areas of real value.



2. System Analysis:

2.1. Requirements Specification:

Specification that are required to detect the image and to rectify

2.1.1.Functional Requirements:

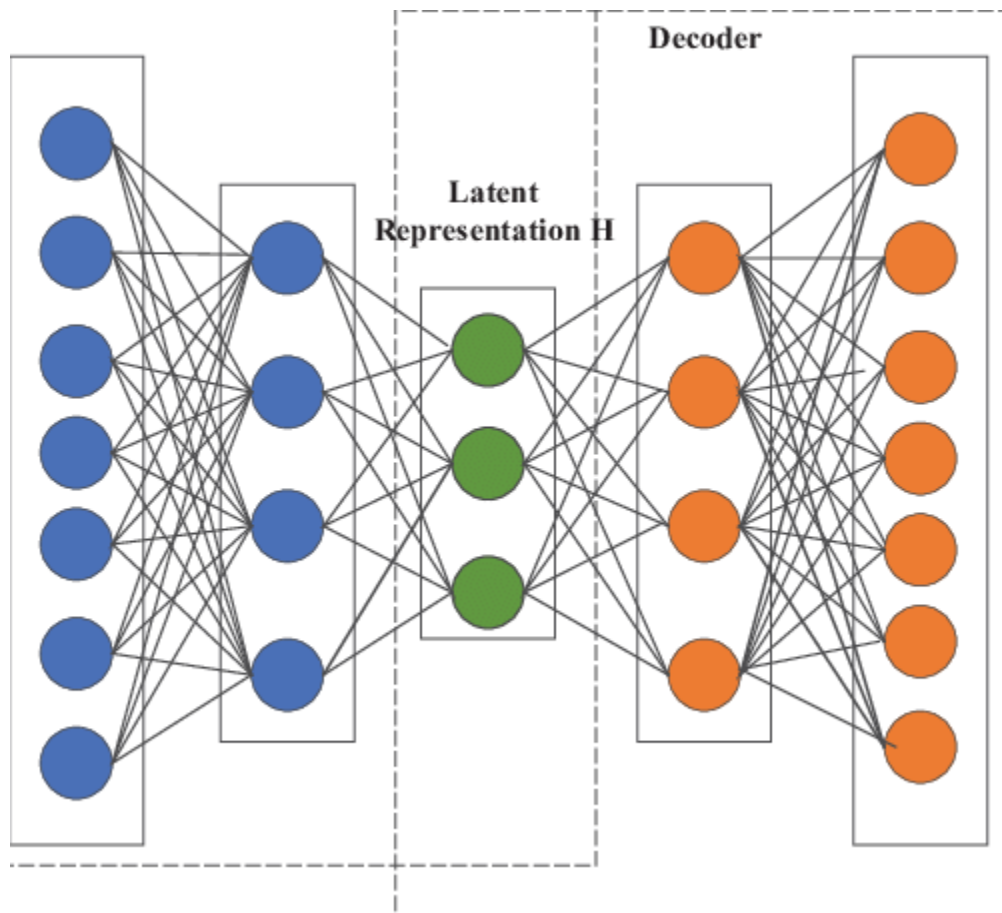
Administrator will carry out these kind of tasks.

- Data-specific: Auto encoders are only able to meaningfully compress data similar to what they have been trained on. Since they learn features specific for the given training data, they are different than a standard data compression algorithm like g zip. So we can't expect an auto encoder trained on handwritten digits to compress landscape photos.
- Lossy : The output of the auto encoder will not be exactly the same as the input, it will be a close but degraded representation. If you want lossless compression they are not the way to go.
- Unsupervised: To train an auto encoder we don't need to do anything fancy, just throw the raw input data at it. Auto encoders are considered an *unsupervised* learning technique since they don't need explicit labels to train on. But to be more precise they are *self-supervised* because they generate their own labels from the training data.

2.1.2. Non- Functional Requirements:

Network Architecture:

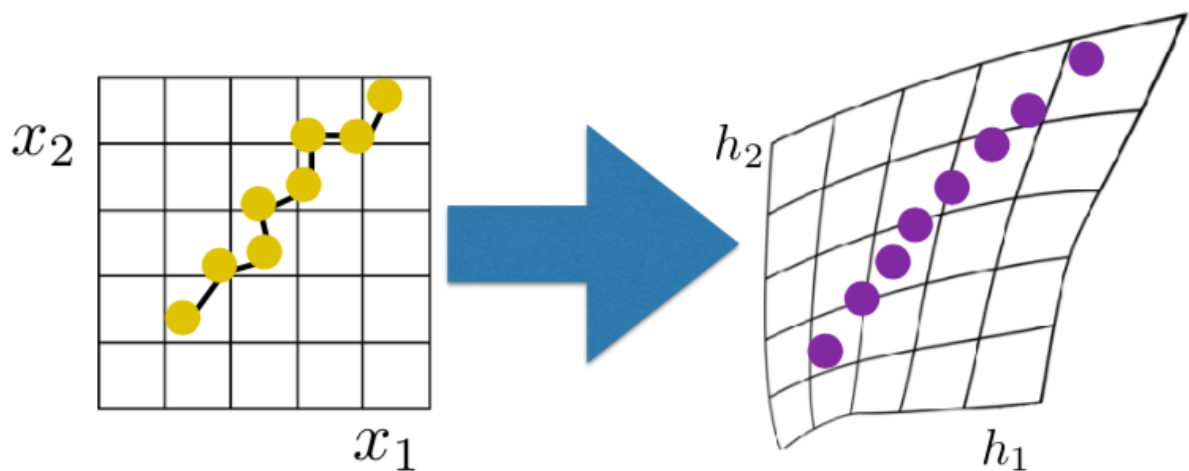
In this stacked architecture, the code layer has a small dimensional value than input information, which is said to be under a complete auto encoder.



Application Architecture:

So how does neural networks solves this problem ? The intuition is, In the manifold space, deep neural networks has the property to bend the space in order to obtain a linear data fold view. Autoencoder architectures applies this property in their hidden layers which allows them to learn low level representations in the latent view space.

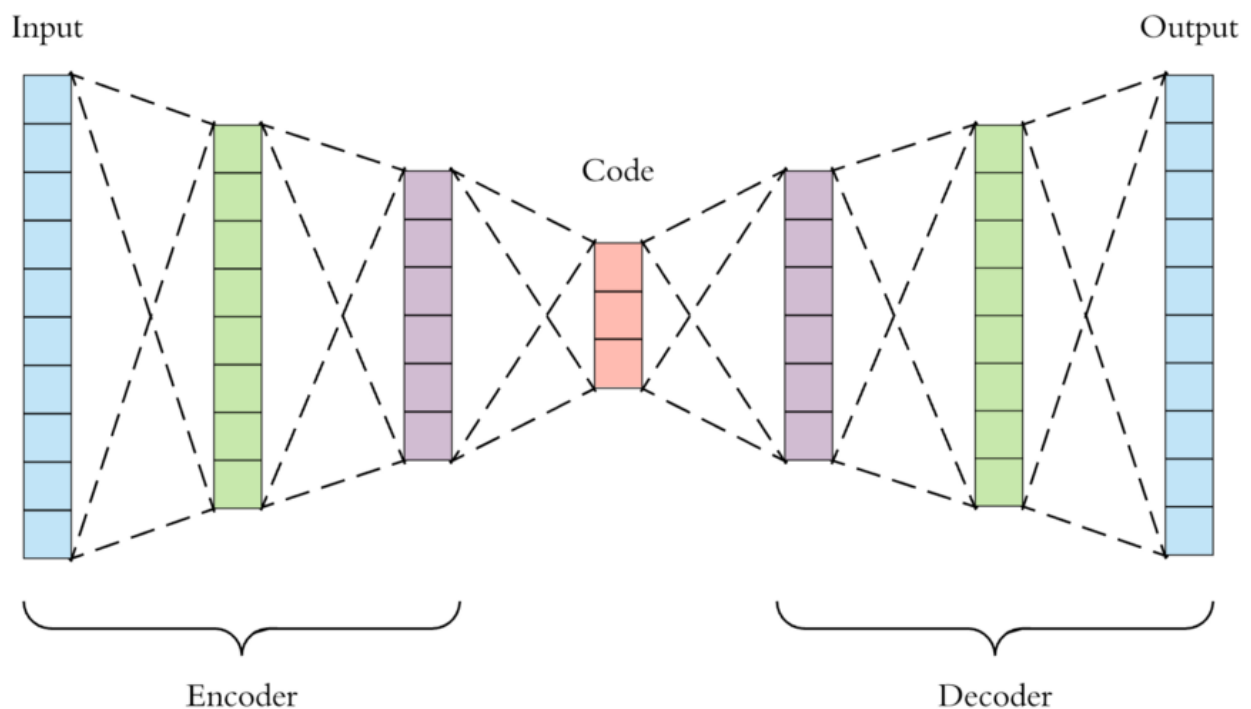
The following image describes this property:



3.SYSTEM DESIGN

3.1. Architecture:

Let's explore the details of the encoder, code and decoder. Both the encoder and decoder are fully-connected feedforward neural networks, essentially the ANNs we covered in [Part 1](#). Code is a single layer of an ANN with the dimensionality of our choice. The number of nodes in the code layer (code size) is a *hyper parameter* that we set before training the auto encoder.



This is a more detailed visualization of an auto encoder. First the input passes through the encoder, which is a fully-connected ANN, to produce the code. The decoder, which has the similar ANN structure, then produces the output only using the code. The goal is to get an output identical with the input. Note that the decoder architecture is the mirror image of the encoder. This is not a requirement but it's typically the case. The only requirement is the dimensionality of the input and output needs to be the same. Anything in the middle can be played with.

There are 4 hyper parameters that we need to set before training an auto encoder:

- Code size: number of nodes in the middle layer. Smaller size results in more compression.
- Number of layers: the auto encoder can be as deep as we like. In the figure above we have 2 layers in both the encoder and decoder, without considering the input and output.
- Number of nodes per layer: the auto encoder architecture we're working on is called a *stacked auto encoder* since the layers are stacked one after another. Usually stacked auto encoders look like a "sand witch". The number of nodes per layer decreases with each subsequent layer of the encoder, and increases back in the decoder. Also the decoder is symmetric to the encoder in terms of layer structure. As noted above this is not necessary and we have total control over these parameters.

- Loss function: we either use *mean squared error (mse)* or *binary cross entropy*. If the input values are in the range [0, 1] then we typically use cross entropy, otherwise we use the mean squared error. For more details check out this [video](#).

3.2. Flow of Work

Objective:

The Primary objective is to build a deep learning model which can identify the blur images in the given data set. Here the accuracy, precision and overall performance can be in the category as follows.

The secondary objective is simply to learn about Simple Auto Encoder, Deep CNN Auto Encoder and De noising Auto Encoder.

Input Data:

To keep things simple, I have lifted a pre-made dataset from kaggle.com.

It's more verbose but a more flexible way to define complex models. We can easily grab parts of our model, for example only the decoder, and work with that. The output of *Dense* method is a callable layer, using the functional API we provide it with the input and store the output. The output of a layer becomes the input of the next layer. With the sequential API the *add* method implicitly handled this for us.

Note that all the layers use the *relu* activation function, as it's the standard with deep neural networks. The last layer uses the *sigmoid* activation because we need the outputs to be between [0, 1]. The input is also in the same range.

Also note the call to fit function, before with ANNs we used to do:

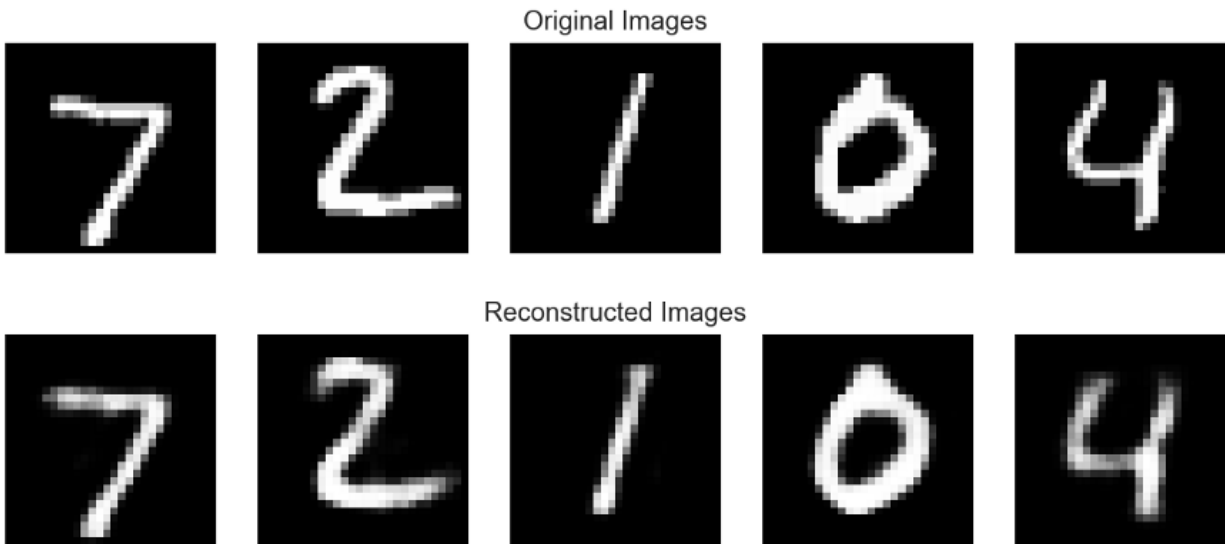
```
model.fit(x_train, y_train)
```

But now we do:

```
model.fit(x_train, x_train)
```

Remember that the targets of the auto encoder are the same as the input. That's why we supply the training data as the target.

We run the auto encoder on the test set simply by using the predict function of Keras. For every image in the test set, we get the output of the auto encoder. We expect the output to be very similar to the input.



4. Testing and Implementation:

- Sequential – This is the Keras Sequential model. It has 3 layers -Embedding Layer, Pooling Layer, and Dense Layer.
- Embedding Layer – The 1st hidden layer of the network. Maps each token to an N-dimensional vector, making a sense that two words with similar meaning tend to have very close vectors.
- Pooling Layer – Converts layer to one dimension by reducing the number of parameters in the model hence helps to avoid overfitting.
- Dense Layer – The final layer that sends the output from the neural network. It has an activation function 'relu' followed by a

dropout layer to avoid overfitting and a final output layer with a sigmoid activation xx function. The sigmoid activation function outputs probabilities between 0(image blur) and 1(optimized image).

I compile the model now! model. compile

(loss='binary_crossentropy', optimizer='adam', metrics= ['accuracy']) • 'binary_crossentropy' – loss function for binary classification.

- 'adam' – optimizer that uses momentum to avoid local minima •
- 'accuracy' – a measure of model performance

Our model is now compiled and ready to be fitted to the training data. Let's fit the model.

```
encoding_dim = 15
input_img = Input(shape=(784,))
# encoded representation of input
encoded = Dense(encoding_dim, activation='relu')(input_img)
# decoded representation of code
decoded = Dense(784, activation='sigmoid')(encoded)
# Model which take input image and shows decoded images
autoencoder = Model(input_img, decoded)
```

Then we need to build the encoder model and decoder model separately so that we can easily differentiate between the input and output.

```
# This model shows encoded images
encoder = Model(input_img, encoded)

# Creating a decoder model
encoded_input = Input(shape=(encoding_dim,))

# last layer of the autoencoder model
decoder_layer = autoencoder.layers[-1]

# decoder model
decoder = Model(encoded_input, decoder_layer(encoded_input))
```

Then we need to compile the model with the ADAM optimizer and cross-entropy loss function fitment.

```
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')
```

Then you need to load the data:

```
(x_train, y_train), (x_test, y_test) = mnist.load_data()

x_train = x_train.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.
```



```
x_train = x_train.reshape((len(x_train), np.prod(x_train.shape[1:])))
x_test = x_test.reshape((len(x_test), np.prod(x_test.shape[1:])))
print(x_train.shape)
print(x_test.shape)
```

It shows like:

Downloading data from

<https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>

11493376/11490434 [=====] -

0s 0us/step

11501568/11490434 [=====] -

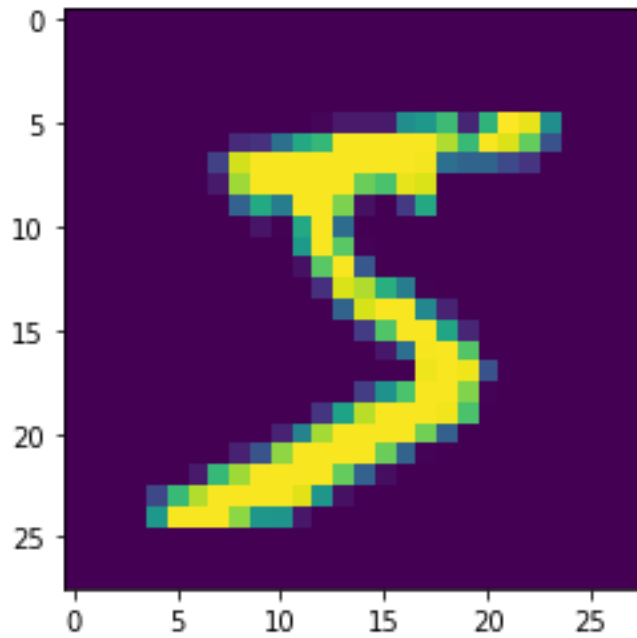
0s 0us/step

(60000, 784)

(10000, 784)

If you want to see how the data is actually, you can use the following line of code :

```
<matplotlib.image.AxesImage at 0x7f1b36851750>
```



Then you need to train your model :

```
autoencoder.fit(x_train, x_train,  
                epochs=15,  
                batch_size=256,  
                validation_data=(x_test, x_test))
```

It then loads and finally we can see the epochs as follows:

Epoch 1/15

235/235 [=====] - 3s 9ms/step - loss:

0.3120 - val_loss: 0.2300

Epoch 2/15

235/235 [=====] - 3s 12ms/step - loss:

0.2066 - val_loss: 0.1868

Epoch 3/15

235/235 [=====] - 2s 9ms/step - loss:

0.1764 - val_loss: 0.1641

Epoch 4/15

235/235 [=====] - 2s 9ms/step - loss:

0.1592 - val_loss: 0.1532

Epoch 5/15

235/235 [=====] - 2s 9ms/step - loss:

0.1515 - val_loss: 0.1477

Epoch 6/15

235/235 [=====] - 2s 9ms/step - loss:

0.1473 - val_loss: 0.1443

Epoch 7/15

235/235 [=====] - 2s 9ms/step - loss:

0.1443 - val_loss: 0.1416

Epoch 8/15

235/235 [=====] - 2s 9ms/step - loss:

0.1418 - val_loss: 0.1393

Epoch 9/15

```
235/235 [=====] - 3s 13ms/step - loss:
0.1397 - val_loss: 0.1374
Epoch 10/15
235/235 [=====] - 2s 10ms/step - loss:
0.1380 - val_loss: 0.1358
Epoch 11/15
235/235 [=====] - 2s 9ms/step - loss:
0.1367 - val_loss: 0.1347
Epoch 12/15
235/235 [=====] - 2s 9ms/step - loss:
0.1357 - val_loss: 0.1338
Epoch 13/15
235/235 [=====] - 2s 9ms/step - loss:
0.1351 - val_loss: 0.1333
Epoch 14/15
235/235 [=====] - 2s 9ms/step - loss:
0.1346 - val_loss: 0.1328
Epoch 15/15
235/235 [=====] - 3s 11ms/step - loss:
0.1342 - val_loss: 0.1324
<keras.callbacks.History at 0x7f1b36346850>
```

So for 15 epochs, it can get maximum image clarification. If u want to have more clarity of your image, then you can load model with more number of epochs.

5.RESULT

After training, you need to provide the input and you can plot the results using the following code :

```
encoded_img = encoder.predict(x_test)
decoded_img = decoder.predict(encoded_img)
plt.figure(figsize=(20, 4))
for i in range(5):
    # Display original
    ax = plt.subplot(2, 5, i + 1)
    plt.imshow(x_test[i].reshape(28, 28))
    plt.gray()
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
    # Display reconstruction
    ax = plt.subplot(2, 5, i + 1 + 5)
    plt.imshow(decoded_img[i].reshape(28, 28))
    plt.gray()
    ax.get_xaxis().set_visible(False)
```

```
ax.get_yaxis().set_visible(False)  
plt.show()
```

Then it can show the result as follows:



So we can clearly see that the blur and original images are there.

We can also do it by other two methods called Deep CNN Auto Encoder and another approach is De noising Auto Encoder. So these also having same approach either.

6.CONCLUSION AND FUTURE WORK

In conclusion, auto encoder is forced to form a representation at the intermediate hidden layer that has a smaller number of variables than the input.

Also we know that auto encoders can give best accuracy in the performance of the train and test data set. This can be concluded by three ways of approach.

The challenges of machine learning algorithms in efficiently handling the threat of spam were pointed out and a comparative study of machine learning techniques available in the literature.

I hope anyone can use this algorithm in order to build a mobile app in which data can be auto detected.

From an auto encoder is a regression task where the network is asked to predict its input (in other words, model the identity function).

We have gone through the structure of how auto encoders work and worked with 3 types of auto encoders. There are multiple uses for auto encoders like dimensionality reduction image compression,

recommendations system for movies and songs and more. The performance of the model can be increased by training it for more epochs or also by increasing the dimension of our network.

7.REFERENCES:

However, the experiment has made efforts towards solving the problem of the detection and also by obtaining the accuracy and performance of less loss. Proposed solutions using legislative, behavioral and technical measures are not a complete solution.

Credit Card Fraud Detection Using Auto encoder model in Unbalanced Datasets and also in the various detections like cancer detection and Anomaly Detection with Auto-Encoders, etc..

Auto encoders are the neural network equivalent of simpler variable compression techniques, like [PCA](#) and [LDA](#). Auto encoders are practically used in deep learning applications for **pre training**: determining the "right" weights for a neural network ahead of time, so that the algorithm doesn't have to work as hard to converge as it would if it had to start with completely random weights. For more on pre training see [my notebook on RBMs](#). In practice, the emergence of [faster convergence algorithms](#) has obviated the need and utility for pre training.

Thus auto encoders are no longer used in cutting edge deep learning work. They're also not notably better than simpler (usually information-theoretic) compression algorithms, like JPEG and MG3, in commercial applications. These days they're most used as a preprocessing step on high-dimensionality data before input to T-SNE.

But they are nevertheless historically and pedagogically important.